

Een if-statement dat je code niet kan schrijven

TypeSafe **Jev**: een "System One"-model dat getypeerde beslissingen geeft, geen tekst

20 minuten · slides + live demo · alles in één marimo-notebook



LIVE API (uit = offline, recordings/)

opnieuw uitvoeren

De hook: LLM-output parsen is fragiel

```
prompt = f"""Classify this ticket as billing, technical or sales.  
Answer with ONLY the label.
```

```
{ticket}"""
```

```
label = llm(prompt).strip().lower()
```

```
if label == "billing":          # "Billing.", "billing (probably)",  
    route_to_billing()        # "I'd say billing", ...
```

```
else:  
    ...                        # en hoe zeker was het model eigenlijk?
```

- Tekst → string-parsing → **retries, regexes, JSON-mode, schema-validators**
- Je krijgt een antwoord, maar **geen kans** dat het klopt
- 1-3 s en een paar cent per beslissing

Wat we eigenlijk willen: $\text{if } P(\text{billing}) > 0.8$:

System 1 vs System 2 (Kahneman)

System 1 · snel, intuïtief

- Herkennen, inschatten, kiezen
- Milliseconden, geen redenering
- *"Dit is een billing-ticket, en dringend"*

→ **Jev** (System One-model)

System 2 · traag, bewust

- Redeneren, plannen, schrijven
- Seconden, stap voor stap
- *"Laat me dit ticket eens goed lezen en een antwoord opstellen"*

→ **LLM** (GPT, Claude, ...)

Beide heb je nodig. De vraag is: **welke beslissingen laat je System 2 nemen die eigenlijk System 1-werk zijn?**

Jev vs LLM

LLM		Jev
Input	prompt (tekst)	state (string/JSON) + getypeeerde vragen
Output	tekst, hopelijk in het juiste formaat	getypeerd antwoord + kansen , schema-conform gegarandeerd
Redenering	chain-of-thought, uitleg	geen (er komt geen reasoning terug)
Latency	1-10 s	70-500 ms (vendor-claim)
Prijs	€ per M tokens in + uit	\$0.042 / M input-tokens, output gratis (vendor)
Goed in	genereren, redeneren	kiezen, scoren, ja/nee — gecalibreerd

Meerdere vragen over dezelfde state gaan in **één call** en worden parallel beantwoord.

Drie primitives + confidence

Confidence = hoe geconcentreerd de verdeling is. Niet: hoe *juist* het antwoord is.

```
QUESTIONS = {
  "team": Choice(
    instructions="Which team should own this ticket?",
    criteria=TEAMS,          # label -> description (or None)
  ),
  "urgency": Score(
    instructions="How urgent is this ticket for the customer?",
    criteria=URGENCY_LEVELS, # ordered rubric: index = score
  ),
  "refund": Noul(
    instructions="Does the customer ask for money back?",
    criteria=REFUND,        # optional: describe yes / no
  ),
}

URGENCY_LEVELS = [
  "No time pressure; can wait a week.", # score 0
  "Should be handled within a few days.", # score 1
  "Blocking the customer; needs a same-day response.", # score 2
]
```

Choice

kies één optie → `.choice`, `.probabilities` (som ≈ 1),
`.confidence`

Score

niveau op een geordende rubriek → `.score` (gewogen),
`.probabilities` per niveau, `.confidence`

Noul

ja/nee → `.noul` = $P(\text{ja})$. Geen aparte confidence: 0.5 betekent *weet het niet*

Demo 1 · Smart if

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output

Hi, I upgraded to the Pro plan on Monday but my invoice shows I was charged twice (EUR 49 and again EUR 49). I need one of those charges reversed. On top of that the export to CSV has been failing all week with a 500 error and my quarterly report is due tomorrow morning, so please look at this today.

```
def smart_if(client: TypeSafeClient, ticket: str) -> SmartIfResult:
    response, latency_ms = timed_system_one(
        client, state={"ticket": ticket}, questions=QUESTIONS
    )
    team = response.choices["team"]          # .choice, .confidence, .probabilities
    urgency = response.scores["urgency"]     # .score, .probabilities per level
    refund = response.nouls["refund"]        # .noul = P(yes)
    return SmartIfResult(
        ticket=ticket,
        team=view(team),
        urgency=view(urgency),
        refund=view(refund),
        meta=CallMeta.of(response, latency_ms),
    )
```

412 ms · één call · model `jev-placeholder` (synthetic, not a real model answer) · 236 input-tokens

billing

team · conf. 0.71

2/2

urgency · Blocking the customer

ja

refund? · P(ja) 0.93

team	P
billing	0.71
technical	0.26
sales	0.02
other	0.01

0 · No time pressure 0.03

1 · Should be handled within a few days. 0.17

2 · Blocking the customer 0.80

Een if-statement dat je code niet kan schrijven

TypeSafe **Jev**: een "System One"-model dat getypeerde beslissingen geeft, geen tekst

20 minuten · slides + live demo · alles in één marimo-notebook



LIVE API (uit = offline, recordings/)

opnieuw uitvoeren

55%
automatisch (6)
45%
escalatie (5)
4.3 s
totale API-tijd · mediaan 388 ms / p95 439 ms

	ticket	team	confidence (drempel 0.75)	urg.	ms	beslissing
T-101	Your API returns 502 on every request since 09:00 UTC. Our...	technical	0.95	2	388	aut
T-102	I was charged EUR 49 twice this month. Please refund one o...	billing	0.96	1	401	aut
T-103	Do you offer a discount for non-profits if we take the ann...	sales	0.91	0	372	aut
T-104	Where can I download my invoices as PDF? I can't find the ...	billing	0.66	0	395	ESCALATI
T-105	The webhook for	technical	0.52	2	431	ESCALATI

	ticket	team	confidence (drempe 0.75)	urg.	ms	beslissin
	'invoice.paid' never fires, so our account...					
T-106	How many seats are included in the Team plan, and can we a...	sales	0.62	0	379	ESCALATI
T-107	Password reset email never arrives (checked spam). Locked ...	technical	0.90	2	366	aut
T-108	Thanks for the great support last week, no action needed!	other	0.94	0	351	aut
T-109	My subscription renewed but the new features from the upgr...	billing	0.49	1	447	ESCALATI
T-110	Can you tell me what the price would be for 250 users, and...	sales	0.84	0	402	aut

	ticket	team	confidence (drempel 0.75)	urg.	ms	beslissing
	T-111	hello?? still waiting	other	0.41	1 344	ESCALATI

Waar Jev in een agent-loop past

user→Jev: routing→LLM plant→Jev: gate→tool→Jev: eval

Routing · welk model / welke agent / welke tool-set?

`ModelRouterMiddleware` in langchain-typesafe **Gates** · mag deze tool call? is dit klaar? moet een mens kijken?

`AutoModeMiddleware` (1 risk-vraag) of je eigen

middleware **Compaction** · welke berichten zijn nog relevant voor de context? **Evals** · is dit antwoord correct / veilig / on-topic? — per turn, in ms Code houdt de controle; Jev levert *programmable common sense*

Demo 3 · Agent gate — Jev keurt elke tool call vooraf

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output

```
def wrap_tool_call(self, request: ToolCallRequest, handler: Callable):
    call = request.tool_call
    body = {"state": gate_state(request), "questions": GATE_QUESTIONS}
    response, ms = timed(lambda: self.classifier.invoke(body))
    p = {name: answer.noul for name, answer in response.nouls.items()}
    blocked, reason = decide(p, self.threshold)
    self.decisions.append(
        GateDecision(call["name"], call["args"], p, blocked, reason, round(ms))
    )
    if blocked:
        return blocked_message(call, reason) # LLM sees an error ToolMessage
    return handler(request) # otherwise: run the tool as usual

def decide(p: dict[str, float], t=THRESHOLD) -> Verdict:
    if p["secrets"] >= t:
        return True, "may expose secrets"
    if p["destructive"] >= t and p["production"] >= t:
        return True, "destructive action on production"
    return False, "allowed"
```

#	tool call	destructief?	productief?
1	run_shell(command='df -h', host='app-01')	0.03	0.0
2	run_sql(query='SELECT query, mean_exec_time A...', database='orders-production')	0.05	0.0
3	run_sql(query='DROP TABLE orders_archive_2023', database='orders-production')	0.97	0.0
4	read_file(path='/etc/app/secrets.env', host='app-01')	0.02	0.0

LLM: placeholder:no-llm-was-called · 4 tool calls, **2 geblokkeerd** · agent-run 14.2 s · tools zijn gesimuleerd, er wordt niets uitgevoerd

Rapport van de agent: 1. Disk on app-01 is at 93% (6 GB free). 2. Slowest query: SELECT * FROM orders WHERE customer_id=? (1840 ms mean, 52k calls). 3. Dropping orders_archive_2023 on orders-production was BLOCKED by the gate (destructive action on production). 4. Reading /etc/app/...

Dezelfde agent, drie poortwachters

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output
+8.8 s

LLM als rechter: extra tijd per run
+1.6 s

Jev als rechter: extra tijd per run
5.6×
gate-tijd LLM / Jev

gate	agent-run	tool calls	geblokkeerd	gate-tijd	kost agent	kost gate
geen gate	12.6 s	4	0	0.0 s	n/a	\$0.000000
LLM als rechter	21.4 s	4	2	9.3 s	n/a	n/a
Jev als rechter	14.2 s	4	2	1.7 s	n/a	\$0.000026

Zelfde taak, zelfde drie vragen, zelfde beleid. De LLM-rechter voegt per tool call een volledige LLM-rondreis toe; Jev een halve seconde en een handvol tokens. **Daarom** zet je met een LLM de check niet op elke stap, en met Jev wel.

Vendor-claims vs wat ik vanuit België mat

Claims van TypeSafe

- latency **70-500 ms**
- input **\$0.042 / M tokens**, output gratis
- *"193.6× sneller / 444.6× goedkoper"* dan een LLM-workflow, op hun **eigen** workflow-evals — zelf noemen ze dat de bovengrens
- service draait aan de **US West Coast**

Gemeten vanuit België

nog niet gemeten (run scripts/benchmark.py)

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output

	demo	n	mediaan	p95	min-max
	demo1 smart if	10	404 ms	487 ms	380-521 ms
	demo2 confidence gate	10	392 ms	440 ms	351-447 ms
	demo3 agent gate	10	414 ms	489 ms	395-512 ms

Demo 1 kost ~236 input-tokens → **\$0.000010** per beslissing.

Amsterdam-Californië alleen al is ~140 ms RTT: de vloer voor EU-gebruikers ligt hoger dan de vendor-vloer.

Head-to-head · dezelfde beslissingen door het LLM en door Jev

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output

6.3×

triage: sneller (mediaan)

n/a

triage: goedkoper

94%

triage: zelfde antwoord (team 82%, urgency 100%, refund 100%)

4.6×

gate: sneller (mediaan)

n/a

gate: goedkoper

94%

gate: zelfde antwoord (destructive 100%, production 100%, secrets 83%)

taak	systeem	n	mediaan	p95	tokens in / uit	kost per 1000
triage	jev	11	388 ms	439 ms	1525 / 0	\$0.0058
triage	llm	11	2460 ms	2820 ms	3065 / 2315	n/a
gate	jev	6	445 ms	490 ms	900 / 0	\$0.0063
gate	llm	6	2050 ms	2312 ms	1800 / 1110	n/a

LLM: placeholder:no-llm-was-called met structured output en **dezelfde criteria-tekst** · Jev: jev-placeholder (synthetic, not a real model answer) · 4 calls parallel · kost: Jev \$0.042 / M

input-tokens (vendor), LLM volgens `LLM_PRICE_*` in `.env` · *zelfde
antwoord* is overeenstemming tussen de twee, geen ground truth

Eén call tegenover twintig samples

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output
T-109 · My subscription renewed but the new features from the upgrade aren't showing. Did the payment go through or is...

team	LLM: aandeel van 20 samples	Jev: kans, 1 call
billing	0.60	0.49
technical	0.35	0.47
sales	0.05	0.02
other	0.00	0.02

60%

LLM eens met zichzelf · zegt zelf 0.88

0.49

Jev confidence · spreiding over 3× 0.00

47 s

20 samples vs 380 ms · n/a vs \$0.000004

T-102 · I was charged EUR 49 twice this month. Please refund one of them.

team	LLM: aandeel van 20 samples	Jev: kans, 1 call
billing	1.00	0.96
technical	0.00	0.02
sales	0.00	0.01
other	0.00	0.01

100%

LLM eens met zichzelf · zegt zelf 0.97

0.96

Jev confidence · spreiding over 3×0.00

47 s

20 samples vs 380 ms · n/a vs \$0.000004

Een LLM-antwoord is **één trekking** uit een verdeling die je nooit ziet; vraag het twintig keer en je reconstrueert ze voor twintig keer de prijs. Jev geeft die verdeling in één call. LLM: `placeholder:no-llm-was-called` · Jev: `jev-placeholder (synthetic, not a real model answer)`

Kanttekeningen

- **Early access:** API en modelnamen kunnen nog bewegen (`jev-latest` is een alias)
- **Vendor-benchmarks:** 193x/444x zijn hun cijfers op hun workflows
- **Latency vanuit de EU:** server aan de US West Coast, reken op de RTT er bovenop
- **Alpha-packages:** `langchain-typesafe` 0.0.1a3, eerste release 17 sept 2026; `typesafe-sdk` 0.7.0
- **Geen reasoning-output:** je krijgt kansen, geen *waarom* — debuggen = state en vragen aanpassen
- **Gecalibreerd $\neq$ correct:** typed output garandeert de interface, niet de waarheid; valideer op je eigen data
- **Confidence** meet concentratie van de verdeling, geen toestemming om te handelen
- Drempels zijn **beleid in code:** evalueer ze op je eigen consequenties

Takeaway

Laat het LLM praten en plannen. Laat System 1 kiezen.

- Beslissingen die nu via *prompt* → *tekst* → *parse* lopen, zijn kandidaten voor **Choice / Score / Noul**
- Kansen + drempel in **jouw** code = een gate die je kan testen, loggen en bijsturen
- Begin met één *smart if* in een bestaande flow; meet latency en kwaliteit op je eigen data

github.com/koenae/jev-demo

alle demo's: `uv run python demos/demo1_smart_if.py --offline`

slides: `uv run marimo run presentation/talk.py`